b230e3.c

b230e3.c > SIZE

```c
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

#define SIZE 100
char stack[SIZE];
int topIndex = -1;

void pushElement(char ch) {
    if (topIndex == SIZE - 1) {
        printf("\nStack Overflow!");
        return;
    }
    stack[++topIndex] = ch;
}

char popElement() {
    if (topIndex == -1) {
        printf("\nStack Underflow!");
        return '\0';
    }
    return stack[topIndex--];
}

char peekElement() {
    if (topIndex == -1) return '\0';
    return stack[topIndex];
}

int precedence(char symbol) {
    if (symbol == '^')
        return 3;
    else if (symbol == '*' || symbol == '/' || symbol == '%')
        return 2;
    else if (symbol == '+' || symbol == '-')
        return 1;
```

b230e3.c

b230e3.c > SIZE

```c
int precedence(char symbol) {
    return 0;
}

void convertInfixToPostfix(char infixExpr[], char postfixExpr[]) {
    int idx = 0, pIdx = 0;
    char currentSymbol;

    while ((currentSymbol = infixExpr[idx++]) != '\0') {
        if (isalnum(currentSymbol)) {
            postfixExpr[pIdx++] = currentSymbol;
        }
        else if (currentSymbol == '(') {
            pushElement(currentSymbol);
        }
        else if (currentSymbol == ')') {
            while (topIndex != -1 && peekElement() != '(') {
                postfixExpr[pIdx++] = popElement();
            }
            if (topIndex != -1 && peekElement() == '(') {
                popElement(); // Remove '(' from stack
            } else {
                printf("\nInvalid Expression: Mismatched Parentheses!");
                exit(1);
            }
        }
        else {
            while (topIndex != -1 && precedence(peekElement()) >= precedence(currentSymbol)) {
                postfixExpr[pIdx++] = popElement();
            }
            pushElement(currentSymbol);
        }
    }

    while (topIndex != -1) {
        if (peekElement() == '(') {
            printf("\nInvalid Expression!");
```

```c
void convertInfixToPostfix(char infixExpr[], char postfixExpr[]) {
    while ((currentSymbol = infixExpr[idx++]) != '\0') {

    while (topIndex != -1) {
        if (peekElement() == '(') {
            printf("\nInvalid Expression!");
            exit(1);
        }
        postfixExpr[pIdx++] = popElement();
    }

    postfixExpr[pIdx] = '\0';
}

int main() {
    char infix[SIZE], postfix[SIZE];

    printf("\nEnter an Infix Expression: ");
    scanf("%s", infix);

    convertInfixToPostfix(infix, postfix);

    printf("\nEquivalent Postfix Expression: %s\n", postfix);

    return 0;
}
```

```
Enter an Infix Expression: A+B-C*D

Equivalent Postfix Expression: AB+CD*-
```